



Parallel Breadth First Search Algorithm

Lightning Presentation

Presented by: Aayush Brahmbhatt & Rahil Shukla

Date: 04/11/2025

**NC STATE
UNIVERSITY**

Motivation & Project Rationale

❖ Why This Topic?

- Scale Challenge: Real-world graphs (e.g., social networks) are too large for efficient sequential processing.
- Application: Used for network analysis, shortest paths, and more.
- Opportunity: Leverage high-performance clusters (ARC cluster) with GPUs and multi-core CPUs.

❖ Goal:

- Compare three parallel BFS strategies (MPI, CUDA, OpenMP).
- Achieve substantial reduction in traversal time over the serial baseline.

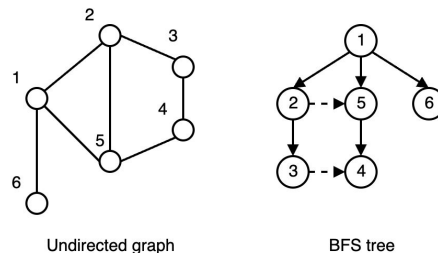
Related Work

- Direction-Optimizing BFS (Beamer et al., 2012)
Switches between top-down and bottom-up strategies to reduce edge scans during large frontier expansions. [1]
- Distributed BFS with 2D Partitioning (Buluc & Madduri, 2016)
Utilizes a 2D graph partitioning scheme to minimize inter-processor communication, forming the basis for our MPI implementation. [2]
- Shared-Memory Optimization (Shun & Blelloch, 2013)
Introduces lightweight techniques for efficient graph traversal on multicore systems, informing our OpenMP approach. [3]
- GPU Graph Processing (Wang et al., 2016)
Demonstrates high-performance BFS on GPUs with optimized memory access and kernel execution, inspiring our CUDA strategy. [4]

Terminology & Serial BFS Overview

- Key Terms:
 - a. Frontier: The set of vertices at the current level of BFS that are being explored.
 - b. Level: Each iteration of BFS, where vertices at the current frontier are expanded to their neighbors.
 - c. Serial BFS:

```
1 function BFS(graph, source)
2   distance[source] = 0
3   currentFrontier = [source]
4   level = 0
5   while currentFrontier is not empty:
6     nextFrontier = []
7     for vertex in currentFrontier:
8       for neighbor in graph.neighbors(vertex):
9         if neighbor not visited:
10          distance[neighbor] = level + 1
11          nextFrontier.append(neighbor)
12     currentFrontier = nextFrontier
13     level++
14
```



Ref: [Udacity](#)

Experimental Setup & Dataset

Dataset: Email-Eu-core Graph [5]

- Vertices: Approximately 1,005
- Edges: Approximately 25,571
- Reason for Choice:
 - ◆ Manageable size for detailed performance analysis.
 - ◆ Real-world network structure with typical low-diameter properties.

Hardware Platform:

- MPI Experiments: Run on 4 nodes (total 16 CPU cores) from the NCSU ARC cluster.
- OpenMP Experiments: Executed on a single node with 16 threads.

Software Environment:

- MPI (using OpenMPI) and OpenMP (using GCC/OpenMP directives) in C/C++

Pseudocode – OpenMP Approach

```
1 function OpenMP_BFS(graph, source)
2     initialize distance[source] = 0
3     currentFrontier = [source]
4     level = 0
5     while currentFrontier is not empty:
6         parallel for each vertex in currentFrontier:
7             for neighbor in graph.neighbors(vertex):
8                 if neighbor not visited:
9                     mark neighbor as visited (level+1)
10                    add neighbor to thread-local queue
11                currentFrontier = merge(thread-local queues)
12                level++
13
```

Pseudocode – MPI Approach

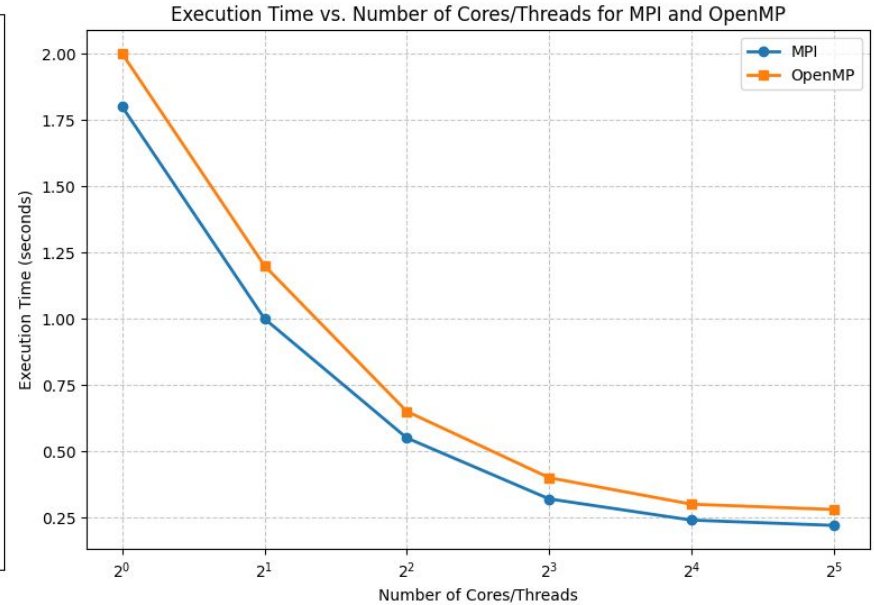
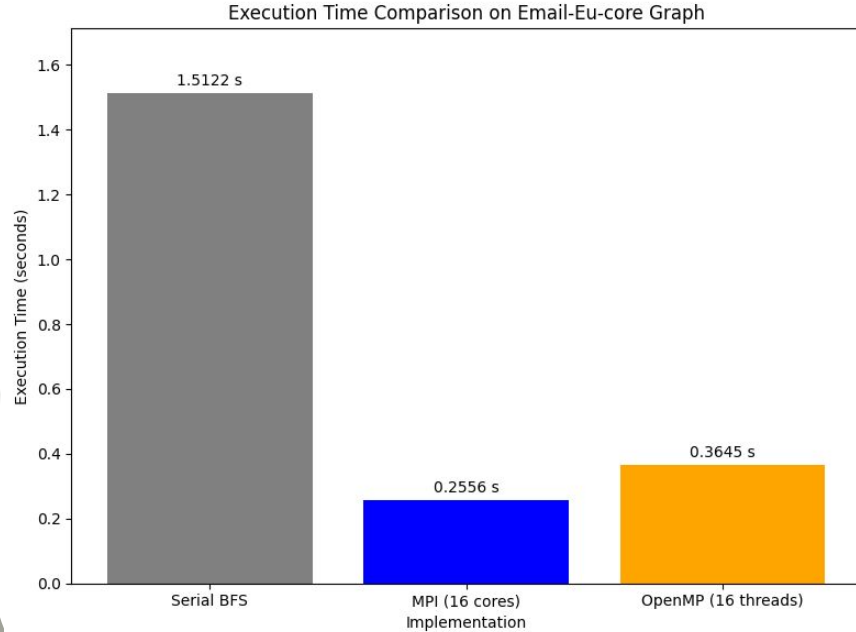
```
1 function MPI_BFS(graph, source)
2   distribute graph among processors in a 2D grid
3   initialize local part: distance[source] = 0 on owning processor
4   localFrontier = [source]
5   while any processor has vertices in its localFrontier:
6     for each processor in parallel:
7       expand localFrontier on local graph partition
8       communicate non-local vertices (intra-row/column)
9   merge results to form global new frontier
```

Pseudocode – CUDA Implementation

```
1- function CUDA_BFS(graph, source)
2-     copy graph data to GPU memory
3-     distance[source] = 0; currentFrontier = [source]; level = 0
4-     while currentFrontier is not empty:
5-         if frontier is small:
6-             launch top-down kernel to process currentFrontier
7-         else:
8-             launch bottom-up kernel where each unvisited vertex checks for a visited neighbor
9-             update currentFrontier with new vertices
10-            level++
```

Direction-Optimizing in progress [1]

Performance Result



Next Steps & QAs

Complete CUDA Implementation:

- ❖ Bottom-Up Approach: [1]
 - Implement and integrate the bottom-up CUDA kernel to efficiently handle large frontiers.

Extended Testing & Profiling:

- ❖ Scale experiments to larger datasets (e.g., Friendster, Twitter network).

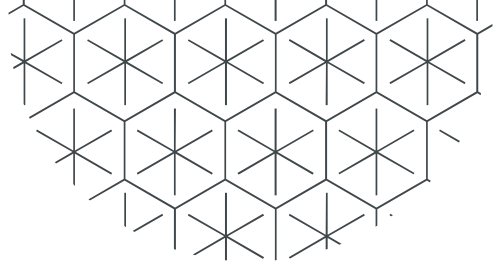
Report Writing

Q&A:

- ❖ Open the floor for any questions.

References

1. Beamer, S., Asanović, K., & Patterson, D. (2012). *Direction-Optimizing Breadth-First Search*. In SC '12: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (pp. 1-10).
2. Buluc, A., & Madduri, K. (2016). *Parallel Breadth-First Search on Distributed Memory Systems*. Proceedings of the International Conference on High Performance Computing, Networking, Storage and Analysis.
3. Shun, J., & Blelloch, G. E. (2013). *Ligra: A Lightweight Graph Processing Framework for Shared Memory*. SIGPLAN Not., 48(8), 135-146.
4. Wang, Y., Davidson, A., Pan, Y., Wu, Y., Riffel, A., & Owens, J. D. (2016). *Gunrock: A High-Performance Graph Processing Library on the GPU*. In Proceedings of the ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming.
5. Leskovec, J., & Krevl, A. (2014). *SNAP Datasets: Stanford Large Network Dataset Collection*. June 2014.



Thank You!